
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 4a: Hardware-Aware Reverse Engineering with CMSIS-SVD: Live GDB and Ghidra Analysis of Stripped Binaries

ARM Cortex-M33 • GDB • Ghidra

September 23, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

By the end of this tutorial, you will be able to:

- Understand the fundamental differences between an ELF binary and a raw `.bin` firmware dump.
- Flash and debug raw binaries using OpenOCD and a hardware Debug Probe with explicit base addressing.
- Operate GDB without an executable or symbol table, using hardware breakpoints (`hb`) and address-based disassembly (`x/i`).
- Navigate the ARM Cortex-M Vector Table at `0x10000000` to find the initial Stack Pointer and Reset Vector.
- Identify the Memory-Mapped I/O (MMIO) peripheral blind spot in stripped binaries.
- Install and configure CMSIS-SVD (`rp2350.svd`) and `PyCortexMDebug` inside GDB for live dynamic peripheral inspection.
- Inspect and manipulate microcontroller hardware registers and bitfields live over SWD directly from the GDB prompt.
- Install and run `SVD-Loader-Ghidra` to map peripheral memory blocks and auto-generate C struct definitions.
- Transform raw, obscure pointer decompilation into readable, vendor-grade peripheral struct accesses.
- Execute a unified hardware-aware reverse engineering workflow combining static analysis in Ghidra and dynamic analysis in GDB.

Part 1: The Raw Binary Dilemma (.elf vs .bin)

ELF Files: The Friendly Development Container

In previous lessons, when debugging your firmware, you launched GDB pointing to an **Executable and Linkable Format (ELF)** file:

```
arm-none-eabi-gdb build\0x0001a_stack.elf
```

An ELF file is not just machine code. It is a rich, structured container that contains:

1. **ELF Header:** Specifies the target architecture, endianness, and exact entry point address.
2. **Section Headers:** Defines memory segments such as `.text`, `.data`, `.rodata`, and `.bss`.
3. **Symbol Table:** Maps human-readable names (`main`, `age`, `gpio_init`) to exact virtual memory addresses.
4. **DWARF Debug Information:** Links individual assembly instructions back to original C source file line numbers, variable types, and stack frame layouts.

When GDB loads an ELF file, it knows the name of every function, the layout of every struct, and the location of `main`.

Raw .bin Files: The Harsh Reality of Firmware Extraction

In real-world hardware reverse engineering, red teaming, and firmware extraction (such as reading an external SPI flash chip or intercepting an over-the-air firmware update), you will almost never have access to an ELF file.

Instead, you are handed a **raw binary file** (`.bin`):

```
+-----+
| Comparison: ELF Container vs. Raw .bin Firmware |
+-----+
| 0x0008_uninitialized-variables.elf              |
+-----+
| | ELF Header (Entry Point: 0x100001cd)          |
| | Symbol Table (main -> 0x10000234, gpio_init -> 0x100002b4) |
| | DWARF Debug Data (C Source Line Mapping)      |
| | Section Table (.text, .rodata, .data, .bss)    |
| | Machine Code Payload                          |
+-----+
| 0x0008_uninitialized-variables.bin              |
+-----+
| | [Flat Machine Code Bytes Only - No Headers, No Symbols] |
| | 00 20 08 20 cd 01 00 10 ...                      |
+-----+
```

A `.bin` file is a byte-for-byte memory dump of flash memory. It contains:

- **No ELF header**
- **No section names**
- **No symbol names**
- **No debug information**
- **No base address metadata** (the file itself does not record where in memory it belongs)

If you attempt to launch GDB with a `.bin` file directly:

```
arm-none-eabi-gdb build\0x0008_uninitialized-variables.bin
```

GDB immediately halts with an error:

```
"build/0x0008_uninitialized-variables.bin": not in executable format: file format not recognized
```

The Hardware MMIO Blind Spot

Stripped binaries introduce a second, even larger obstacle: **hardware peripherals**.

Microcontrollers interact with the outside world using **Memory-Mapped Input/Output (MMIO)**.

Peripherals like GPIO, UART, SPI, and Clocks do not have special CPU instructions. Instead, they are mapped to specific, fixed physical addresses in the microcontroller's memory map:

Subsystem	RP2350 Physical Address Base	Function
IO_BANK0	0x40028000	GPIO pin function select (FUNCSEL) and overrides
PADS_BANK0	0x40038000	Electrical pad controls (drive strength, pulls, Schmitt)
SI0	0xd0000000	Single-cycle I/O (high-speed GPIO set, clear, and toggle)

When analyzing a raw binary, neither GDB nor Ghidra understands what `0x40038044` or `0xd0000014` mean. To standard tools, they are just arbitrary hexadecimal numbers. Reverse engineers are left manually cross-referencing thousands of pages of microcontroller datasheets.

In this tutorial, you will master the two industry-standard tools that eliminate this blind spot:

1. **PyCortexMDebug** for live, dynamic hardware awareness in **GDB**.
2. **SVD-Loader-Ghidra** for automatic struct mapping and clear decompilation in **Ghidra**.

Part 2: Reviewing the Week 4 Target Programs

In Week 4, we worked with two distinct programs:

1. `0x0005_intro-to-variables.c` : Explored initialized variable allocation, assignments, and string printing.
2. `0x0008_uninitialized-variables.c` : Explored uninitialized variable behavior in `.bss` and introduced real-time **GPIO hardware peripheral control**.

Let's review both source files from your workspace:

Program 1: `0x0005_intro-to-variables.c`

```
#include <stdio.h>
#include "pico/stdlib.h"

int main(void) {
    uint8_t age = 42;

    age = 43;

    stdio_init_all();

    while (true)
        printf("age: %d\r\n", age);
}
```

This first program taught us how variables are assigned registers and stack slots, but it only interacted with the console output.

Program 2: `0x0008_uninitialized-variables.c`

To study hardware peripherals and Memory-Mapped I/O (MMIO), we turn to our primary target for this supplement:

```
#include <stdio.h>
#include "pico/stdlib.h"

#define LED_PIN 16

int main(void) {
    uint8_t age; // Uninitialized!

    stdio_init_all();

    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);

    while (true) {
        printf("age: %d\r\n", age);

        gpio_put(LED_PIN, 1);
        sleep_ms(500);

        gpio_put(LED_PIN, 0);
        sleep_ms(500);
    }
}
```

Hardware Actions Performed by This Code

1. `stdio_init_all()` : Configures the UART/USB subsystem to stream serial data.
2. `gpio_init(LED_PIN)` :
 - Accesses `PADS_BANK0` register `0x40038044` to enable input/output pads.
 - Accesses `IO_BANK0` register `0x40028084` to set `FUNCSEL` to 5 (SIO).
3. `gpio_set_dir(LED_PIN, GPIO_OUT)` :
 - Configures the Output Enable bit in SIO or via the RP2350 GPIO coprocessor.
4. `gpio_put(LED_PIN, 1)` **and** `gpio_put(LED_PIN, 0)` :
 - Writes to SIO register `0xd0000014` (`GPIO_OUT_SET`) to turn on the LED.
 - Writes to SIO register `0xd0000018` (`GPIO_OUT_CLR`) to turn off the LED.

When compiled, the build system produces:

- `build/0x0008_uninitialized-variables.elf` (Full ELF with debug symbols)
- `build/0x0008_uninitialized-variables.bin` (Raw stripped machine code payload)
- `build/0x0008_uninitialized-variables.uf2` (Packaged binary for USB drag-and-drop flashing)

For the remainder of this lesson, we assume you **only have** `0x0008_uninitialized-variables.bin`.

Part 3: Flashing and Connecting OpenOCD with Raw Binaries

Flashing a Raw Binary via OpenOCD

When flashing an `.elf` file, OpenOCD reads the target memory addresses directly from the ELF program headers. However, because a `.bin` file contains no header information, you **must explicitly specify the physical base address** (`0x10000000` for RP2350 XIP flash).

Open a PowerShell terminal in the project directory:

```
& "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\openocd.exe" `
-s "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\scripts" `
-f interface/cmsis-dap.cfg `
-f target/rp2350.cfg `
-c "adapter speed 5000; program build/0x0008_uninitialized-variables.bin 0x10000000 verify reset
exit"
```

Notice the crucial parameter: `0x10000000`. This instructs OpenOCD to write the binary bytes starting at the exact beginning of external flash memory.

```
** Programming Started **
[rp2350.dap.core0] target halted due to debug-request
wrote 32768 bytes from file build/0x0008_uninitialized-variables.bin in 0.812s
** Programming Finished **
** Verify Started **
verified 32768 bytes in 0.142s
** Verified OK **
** Resetting Target **
shutdown command invoked
```

Starting OpenOCD as a Live Debug Server

To debug the running firmware interactively, launch OpenOCD without the `exit` command. Keep this terminal open:

```
& "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\openocd.exe" `
-s "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\scripts" `
-f interface/cmsis-dap.cfg `
-f target/rp2350.cfg `
-c "adapter speed 5000; init"
```

OpenOCD initializes the SWD hardware connection and listens for GDB connections on TCP port `3333`.

```
Info : Hardware thread awareness created
Info : Listening on port 3333 for gdb connections
```

Part 4: Dynamic Analysis in GDB Without an ELF

Because we are analyzing a raw binary, we do not pass a file name to GDB on startup.

Open a second terminal window:

```
arm-none-eabi-gdb
```

Step 1: Set Target Architecture and Connect

Inside the GDB prompt, tell GDB what processor architecture to expect, connect over TCP to OpenOCD, and immediately halt the processor:

```
set architecture armv8-m.main
target extended-remote :3333
monitor reset halt
```

```
The target architecture is assumed to be armv8-m.main
Remote debugging using :3333
target halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0x100001cc msp: 0x20082000
```

Step 2: Decode the Hardware Vector Table

How did the CPU know to halt at `0x100001cc`?

On ARM Cortex-M processors, the first entries of flash memory (`0x10000000`) define the **Vector Table**:

- **Word 0** (`0x10000000`): Initial Main Stack Pointer (`msp`).
- **Word 1** (`0x10000004`): Reset Vector (address of the first instruction to run upon reboot).

Let's inspect the first two 32-bit words of flash using GDB's examine command (`x/2wx`):

```
x/2wx 0x10000000
```

```
0x10000000: 0x20082000 0x100001cd
```

Let's analyze what these two words reveal:

```
+-----+
| RP2350 Cortex-M33 Vector Table Decoding |
+-----+
| Address      Value      Meaning          |
| 0x10000000:  0x20082000  Initial MSP (Top of 512KB SRAM) |
| 0x10000004:  0x100001cd  Reset Vector (Thumb Execution Bit 0) |
|                                     Execution begins at: 0x100001cc |
+-----+
```

[!NOTE] The Reset Vector value is `0x100001cd`. In ARM architecture, bit 0 indicates Thumb instruction mode (`0x100001cc + 1`). The processor automatically clears bit 0 and begins executing Thumb instructions at address `0x100001cc`.

Step 3: Why Symbolic Commands Fail

If you try to use symbolic commands, GDB cannot help you:

```
break main
disassemble main
```

```
No symbol table is loaded. Use the "file" command.
```

There are no symbols! To reverse engineer this firmware dynamically, we must navigate using **instruction addresses**.

Step 4: Disassemble by Address Range

Disassemble the 10 instructions starting at the current Program Counter (`$pc`):

```
x/10i $pc
```

```
=> 0x100001cc: ldr    r0, [pc, #36]    ; 0x100001f4
0x100001ce: msr    MSP, r0
0x100001d2: bl     0x1000019a
0x100001d6: bl     0x10000210
0x100001da: bl     0x10000234
0x100001de: b.n    0x100001de
```

Look at line `0x100001da` :

```
0x100001da: bl     0x10000234
```

This branch with link (`bl`) calls subroutine `0x10000234` . From our firmware structure, this is `main()` !

Step 5: Setting Hardware Breakpoints on Flash Memory

In RAM, GDB can set software breakpoints by temporarily replacing instructions with a breakpoint opcode (`bkpt`). However, external XIP flash (`0x10000000` - `0x1FFFFFFF`) is **read-only**. GDB cannot write to flash memory while the processor is running.

Therefore, you must use **hardware breakpoints** (`hb`):

```
hb *0x10000234
continue
```

```
Hardware assisted breakpoint 1 at 0x10000234
Continuing.
```

```
Breakpoint 1, 0x10000234 in ?? ()
```

We are now stopped at the entry point of `main()` inside a completely stripped binary!

Disassemble the instructions in `main()` :

```
disassemble 0x10000234, 0x10000260
```

```
Dump of assembler code from 0x10000234 to 0x10000260:
=> 0x10000234: push    {r4, lr}
0x10000236: bl     0x1000030cc
0x1000023a: movs   r0, #16
0x1000023c: bl     0x100002b4
0x10000240: movs   r1, #1
0x10000242: movs   r0, #16
0x10000244: bl     0x100002e0
End of assembler dump.
```

Notice:

- `0x1000023a`: `movs r0, #16` followed by `bl 0x100002b4` -> This is `gpio_init(16)` !
- `0x10000240`: `movs r1, #1` and `movs r0, #16` followed by `bl 0x100002e0` -> This is `gpio_set_dir(16, GPIO_OUT)` !

Step 6: The MMIO Blindness in Action

Step into `gpio_init` or inspect the peripheral registers directly.

In standard GDB, if you want to inspect what happened to GPIO pin 16, you are forced to type raw hex addresses:

```
x/wx 0x40038044
```

```
0x40038044: 0x0000005a
```

What does `0x5a` mean?

- Is Output Disable set?
- Is Input Enable active?
- Is the pull-up enabled?
- What is the drive strength?

Standard GDB has no way to tell you. Let's fix that right now.

Part 5: Installing and Configuring CMSIS-SVD in GDB (PyCortexMDebug)

What is CMSIS-SVD?

CMSIS-SVD (Common Microcontroller Software Interface Standard - System View Description) is an open XML specification developed by ARM. Silicon vendors publish an `.svd` file for every chip they manufacture.

An SVD file contains a complete, machine-readable description of:

- Every peripheral on the chip
- Its physical base address
- Every register and offset
- Every bitfield, bitmask, access permission, and human-readable description

```

+-----+
| CMSIS-SVD Hierarchy Tree |
+-----+
| Device: RP2350           |
| +-- Peripheral: SIO (Base: 0xd0000000) |
| |   +-- Register: GPIO_OUT (Offset: 0x010) |
| |   +-- Register: GPIO_OUT_SET (Offset: 0x014) |
| |   +-- Register: GPIO_OUT_CLR (Offset: 0x018) |
| +-- Peripheral: PADS_BANK0 (Base: 0x40038000) |
| |   +-- Register: GPIO16 (Offset: 0x044) |
| |       +-- Bitfield: OD (Bit 7: Output Disable) |
| |       +-- Bitfield: IE (Bit 6: Input Enable) |
| |       +-- Bitfield: DRIVE (Bits 5..4: Drive Strength) |
| |       +-- Bitfield: PUE (Bit 3: Pull Up Enable) |
| |       +-- Bitfield: PDE (Bit 2: Pull Down Enable) |
+-----+

```

Step 1: Download `rp2350.svd`

Download the official `rp2350.svd` file directly from the open-source `cmsis-svd-data` repository.

Open a PowerShell terminal and create a dedicated `svd` directory in your user profile:

```

New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\svd"
Invoke-WebRequest `
  -Uri "https://raw.githubusercontent.com/cmsis-svd/cmsis-svd-
  data/main/data/RaspberryPi/rp2350.svd" `
  -OutFile "$env:USERPROFILE\svd\rp2350.svd"

```

Verify that the file is downloaded:

```
Get-Item "$env:USERPROFILE\svd\rp2350.svd"
```

Step 2: Install `PyCortexMDebug`

`PyCortexMDebug` is an open-source Python extension for GDB created by Bill Nahill. It parses SVD XML files and exposes high-level peripheral inspection commands inside your GDB session.

Clone the repository into your user profile:

```

cd "$env:USERPROFILE"
git clone https://github.com/bnahill/PyCortexMDebug.git

```

Step 3: Load `PyCortexMDebug` in GDB

In your active GDB session, load the Python script and parse `rp2350.svd`:

```

source ~/PyCortexMDebug/scripts/gdb.py
svd_load ~/.svd/rp2350.svd

```

```

Loading SVD file /Users/username/.svd/rp2350.svd...
Loaded 52 peripherals

```

Just like that, GDB now understands every single register, offset, and bitfield on the RP2350 microcontroller!

[!TIP] You can automate this process so SVD support is always active. Add the following lines to your `~/.gdbinit` file:

```
source ~/PyCortexMDebug/scripts/gdb.py
svd_load ~/.svd/rp2350.svd
```

Part 6: Live Dynamic Peripheral Inspection & Manipulation in GDB

Now that GDB is hardware-aware, let's explore what we can do on our stripped `.bin` firmware without touching source code.

Command 1: List All On-Chip Peripherals

Type `svd` with no arguments to list all 52 peripherals on the RP2350:

```
svd
```

Available Peripherals:

ACCESSCTRL	ADC	BUSCTRL	CLOCKS
DMA	GLITCH_DETECTOR	I2C0	I2C1
IO_BANK0	IO_QSPI	OTP	PADS_BANK0
PADS_QSPI	PIX_RP2040	PI00	PI01
PI02	PLL_SYS	PLL_USB	POWMAN
PWM	QMI	RESETS	ROSC
SHA256	SIO	SPI0	SPI1
SYSINFO	SYSCFG	TBMAN	TICKS
TIMER0	TIMER1	TRNG	UART0
UART1	USB	VREG_AND_CHIP_RESET	WATCHDOG
XIP_AUX	XIP_CTRL	XIP_QMI	XOSC

Command 2: Inspect a Peripheral Live (`svd SIO`)

Inspect the **SIO (Single-Cycle I/O)** peripheral. GDB automatically reads the physical hardware registers across SWD and displays their live state:

```
svd SIO
```

SIO @ 0xd0000000:

```
CPUID      : 0x00000000
GPIO_IN    : 0x00000000
GPIO_IN_HI : 0x00000000
GPIO_OUT   : 0x00010000
GPIO_OUT_SET : 0x00000000
GPIO_OUT_CLR : 0x00000000
GPIO_OUT_XOR : 0x00000000
GPIO_OE    : 0x00010000
GPIO_OE_SET : 0x00000000
GPIO_OE_CLR : 0x00000000
```

Look at `GPIO_OUT` and `GPIO_OE`:

- Value: `0x00010000` = Bit 16 is high ($1 \ll 16$)!
- This immediately confirms that **GPIO 16 output is enabled and driven HIGH**.

Command 3: Decode Bitfields (`svd /r PADS_BANK0 GPIO16`)

Remember the mysterious `0x0000005a` we saw when reading `0x40038044` ? Let's use the `/r` (raw/register decode) flag to decode it:

```
svd /r PADS_BANK0 GPIO16
```

```
PADS_BANK0.GPIO16 @ 0x40038044: 0x0000005a
[7]  OD      : 0 (Output Disable: Output enabled)
[6]  IE      : 1 (Input Enable: Input enabled)
[5:4] DRIVE  : 1 (Drive strength: 4mA)
[3]  PUE     : 1 (Pull-up: Enabled)
[2]  PDE     : 0 (Pull-down: Disabled)
[1]  SCHMITT : 1 (Schmitt trigger: Enabled)
[0]  SLEWFAST : 0 (Slew rate: Slow)
```

In a single command, GDB completely decoded the raw hardware byte into human-readable electrical configuration!

Command 4: Live Hardware Manipulation via SWD (`svd /w`)

You can also **write** to registers by name using `svd /w`.

While the processor is halted in GDB, look at your physical Pico 2 hardware board. Let's turn off the blinking LED by writing directly to `SI0.GPIO_OUT_CLR` :

```
svd /w SI0 GPIO_OUT_CLR 0x00010000
```

The red LED on GPIO 16 instantly turns OFF!

Now turn it back ON:

```
svd /w SI0 GPIO_OUT_SET 0x00010000
```

The red LED on GPIO 16 turns ON!

You have full, symbolic, hardware-level control over a running target directly through GDB—without having an ELF file or compiling a single line of code.

Part 7: Transitioning to Static Analysis in Ghidra

Dynamic analysis in GDB lets us verify hardware state while stepping through execution. However, to understand the overall architecture, algorithms, and logic of a stripped firmware binary, we must perform **static analysis in Ghidra**.

Step 1: Import the Stripped Binary into Ghidra

1. Launch Ghidra: `ghidraRun`
2. Create a new project named `0x0008_uninitialized-variables`
3. Drag and drop `build/0x0008_uninitialized-variables.bin` into the Active Project window.
4. In the Import dialog:
 - **Language:** Click `...`, search for `Cortex`, and select **ARM:LE:32:Cortex (default)**.
 - Click **Options...**:
 - Change **Block Name** to `.text`
 - Change **Base Address** to `10000000` (XIP Flash base)
 - Click **OK**, then click **OK** to complete the import.
5. Double-click the file to open the CodeBrowser.
6. When prompted to analyze, click **Yes**, accept default analyzers, and click **Analyze**.

Step 2: The Decompiler Before SVD (The Pointer Maze)

Navigate to `main` at address `0x10000234` in the Ghidra decompiler.

Before adding SVD support, the decompiler output looks like this:

```
void FUN_10000234(void)
{
    undefined4 extraout_r1;
    undefined4 extraout_r2;
    undefined4 in_cr0;
    undefined4 in_cr4;

    FUN_100030cc();
    FUN_100002b4(0x10);
    coprocessor_moveto2(0,4,0x10,1,in_cr4);
    do {
        FUN_1000325c(DAT_10000274,0);
        coprocessor_moveto2(0,4,0x10,1,in_cr0);
        FUN_10000d10(500);
        coprocessor_moveto2(0,4,0x10,0,in_cr0);
        FUN_10000d10(500,extraout_r1,extraout_r2,0);
    } while( true );
}
```

Now navigate to subroutine `FUN_100002b4 (gpio_init)`:

```
void FUN_100002b4(uint param_1)
{
    *(uint*)(0x40038000 + param_1 * 4 + 4) =
        *(uint*)(0x40038000 + param_1 * 4 + 4) & 0xffffffff7f | 0x40;
    *(uint*)(0x40028000 + param_1 * 8 + 4) = 5;
    return;
}
```

Look at that decompilation:

- `*(uint*)(0x40038000 + param_1 * 4 + 4)`
- `*(uint*)(0x40028000 + param_1 * 8 + 4) = 5`

Ghidra doesn't know that `0x40038000` is `PADS_BANK0`, nor that `0x40028000` is `IO_BANK0`. In fact, if you double-click `0x40038000`, Ghidra warns that the address is **unmapped** because the `.bin` import only created a memory block for flash (`0x10000000`).

Part 8: Installing and Running SVD-Loader in Ghidra

Step 1: Install SVD-Loader-Ghidra

`SVD-Loader-Ghidra` is an open-source Ghidra script created by Leveldown Security. It parses CMSIS-SVD files and automatically reconstructs microcontroller memory maps and data structures inside Ghidra.

Open a PowerShell terminal and clone the repository:

```
cd "$env:USERPROFILE"
git clone https://github.com/leveldown-security/SVD-Loader-Ghidra.git
```

Step 2: Add the Script to Ghidra Script Manager

1. In Ghidra's CodeBrowser, open the **Script Manager**:
 - Click the green "Play" icon on the toolbar, or go to **Window -> Script Manager**.
2. Click the **Script Directories** icon (the small folder icon in the top right of the Script Manager toolbar).
3. Click the green **+** icon to add a new directory.
4. Browse to and select: `C:\Users\<username>\SVD-Loader-Ghidra`
5. Click **OK**.

Step 3: Run SVD-Loader

1. In the Script Manager search filter box, type: `SVD`
2. Select `SVD-Loader.py` from the list.
3. Click the green **Run Script** button in the top right.
4. A file picker dialog opens:
 - Navigate to: `C:\Users\<username>\.svd\rp2350.svd`
 - Click **Open**.

What SVD-Loader Does Automatically in Ghidra

1. Memory Blocks: Creates mapped, volatile memory blocks for `IO_BANK0`, `PADS_BANK0`, `SIO`, `UART0`, `CLOCKS`, and `RESETS`.
2. Symbol Labels: Creates global symbol labels at the exact address of every register (e.g., `SIO_GPIO_OUT_SET`).
3. C Structs: Generates full peripheral data structures in the Data Type Manager (e.g., `struct SIO_Type`).

Check the Ghidra Console window at the bottom of the screen. You will see:

```

Loaded SVD: rp2350.svd
Created peripheral block: SIO at 0xd0000000 (size: 0x1000)
Created peripheral block: PADS_BANK0 at 0x40038000 (size: 0x1000)
Created peripheral block: IO_BANK0 at 0x40028000 (size: 0x1000)
...
Successfully imported all peripherals!

```

Part 9: Decompiler Transformation: Before and After

Now that the SVD structures are loaded, let's examine subroutine `FUN_100002b4` (`gpio_init`) again.

Side-by-Side Decompilation Comparison

```

+-----+
| Decompilation of gpio_init()
|
| BEFORE SVD-Loader:
| void FUN_100002b4(uint param_1)
| {
|     *(uint *)(0x40038000 + param_1 * 4 + 4) =
|         *(uint *)(0x40038000 + param_1 * 4 + 4) & 0xffffffff7f
|         | 0x40;
|     *(uint *)(0x40028000 + param_1 * 8 + 4) = 5;
|     return;
| }
|
| AFTER SVD-Loader:
| void gpio_init(uint gpio)
| {
|     PADS_BANK0->GPIO[gpio] =
|         (PADS_BANK0->GPIO[gpio] & ~PADS_BANK0_OD)
|         | PADS_BANK0_IE;
|     IO_BANK0->GPIO[gpio].CTRL = IO_BANK0_FUNCSEL_SIO; /* 5 */
|     return;
| }
+-----+

```

Look at the difference:

1. `*(uint *)(0x40038000 + param_1 * 4 + 4)` is now recognized as indexing into `PADS_BANK0->GPIO[gpio]`.
2. `& 0xffffffff7f | 0x40` is clearly revealed as clearing the `OD` (Output Disable) bit and setting the `IE` (Input Enable) bit.
3. `*(uint *)(0x40028000 + param_1 * 8 + 4) = 5` is recognized as setting the pin's multiplexer control register (`CTRL`) to function select `5` (`SIO`).

Exploring Structs in the Data Type Manager

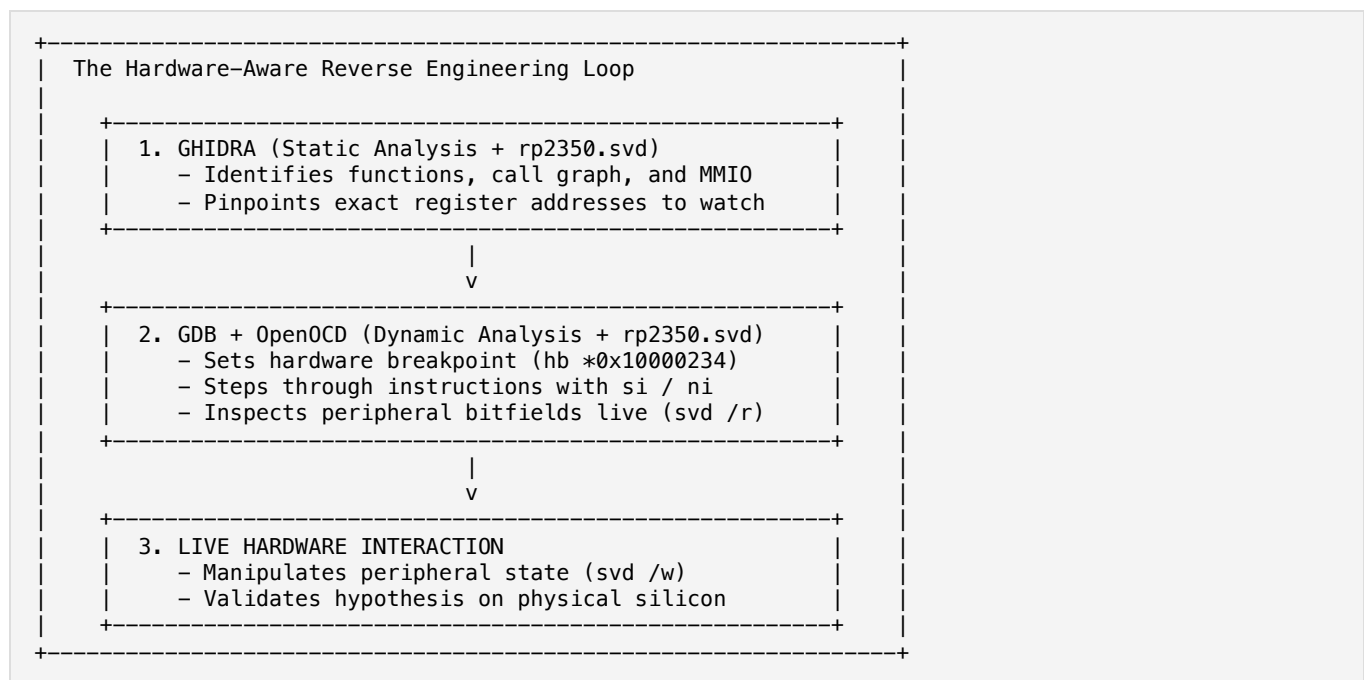
In Ghidra's **Data Type Manager** panel (bottom-left):

1. Expand the tree node for `0x0008_uninitialized-variables.bin`.
2. Expand the `rp2350.svd` category.
3. Locate `SI0_Type` :
 - Double-click `SI0_Type` to open Ghidra's Structure Editor.
 - You can see every field, its byte offset, and its data type:
 - `0x010: GPIO_OUT (uint32_t)`
 - `0x014: GPIO_OUT_SET (uint32_t)`
 - `0x018: GPIO_OUT_CLR (uint32_t)`
 - `0x020: GPIO_OE (uint32_t)`

You can apply these struct types to any pointer in Ghidra by right-clicking a variable in the decompiler and selecting **Retype Variable** -> `SI0_Type *`.

Part 10: The Complete Hardware-Aware Reverse Engineering Workflow

By combining SVD in both GDB and Ghidra, you achieve a seamless reverse engineering loop:



Part 11: Summary, Cheatsheets & Review

GDB Raw Binary Debugging Cheatsheet

Task	Command	Description
Set Architecture	<code>set architecture armv8-m.main</code>	Configures GDB for ARM Cortex-M33 cores.

Task	Command	Description
Connect to OpenOCD	<code>target extended-remote :3333</code>	Connects to OpenOCD debug server.
Reset & Halt	<code>monitor reset halt</code>	Sends reset signal and halts CPU at vector table.
Read Vector Table	<code>x/2wx 0x10000000</code>	Displays Initial Stack Pointer and Reset Vector.
Disassemble at PC	<code>x/10i \$pc</code>	Disassembles 10 instructions at current Program Counter.
Disassemble Range	<code>disassemble 0x10000234, 0x10000260</code>	Disassembles instructions between two hex addresses.
Hardware Breakpoint	<code>hb *0x10000234</code>	Sets hardware breakpoint on read-only flash memory.
Read Memory Word	<code>x/wx 0x40038044</code>	Reads one 32-bit hexadecimal word from memory.
Write Memory Word	<code>set *0xd0000014 = 0x10000</code>	Writes 32-bit value directly to memory address.

PyCortexMDebug SVD Cheatsheet

Task	Command	Description
Load Plugin	<code>source ~/PyCortexMDebug/scripts/gdb.py</code>	Imports PyCortexMDebug into GDB Python engine.
Load SVD File	<code>svd_load ~/.svd/rp2350.svd</code>	Parses chip peripheral XML definition.
List Peripherals	<code>svd</code>	Lists all on-chip hardware peripheral blocks.
Dump Peripheral	<code>svd SIO</code>	Reads and displays all registers in a peripheral.
Decode Bitfields	<code>svd /r PADS_BANK0 GPIO16</code>	Decodes individual bitfields and named flags.
Write Register	<code>svd /w SIO GPIO_OUT_SET 0x10000</code>	Writes to peripheral register by symbolic name.

RP2350 Peripheral Memory Map Quick Reference

Peripheral	Base Address	Size	Primary Purpose
XIP_FLASH	0x10000000	Up to 16MB	External QSPI Flash execution memory
SRAM	0x20000000	512KB	On-chip data memory (stack, heap, .data, .bss)
IO_BANK0	0x40028000	4KB	GPIO pin function multiplexing (FUNCSEL)
PADS_BANK0	0x40038000	4KB	Electrical drive strength, pulls, and enables
UART0	0x40070000	4KB	Serial communication interface 0
UART1	0x40078000	4KB	Serial communication interface 1
SIO	0xd0000000	4KB	Single-cycle I/O fast GPIO controls

Key Takeaways

1. **A raw `.bin` is not an ELF:** It has no headers, no symbols, and no entry point metadata. You must supply the base address (`0x10000000`) when flashing and debug using explicit addresses.
2. **Flash requires hardware breakpoints:** External flash is read-only during execution. Always use `hb *address` instead of software breakpoints (`b`).
3. **The Vector Table tells all:** Even without symbols, `x/2wx 0x10000000` gives you the initial stack pointer and the reset vector within seconds.
4. **SVD bridges the hardware gap:** CMSIS-SVD turns raw hexadecimal registers into named peripherals, registers, and bitfields across both GDB and Ghidra.
5. **GDB handles live dynamic manipulation:** `PyCortexMDebug` lets you inspect peripheral registers live and toggle pins directly over SWD using `svd /w`.
6. **Ghidra handles static comprehension:** `SVD-Loader-Ghidra` creates mapped memory blocks and C structs, converting raw pointer arithmetic into readable code.

Glossary

Term	Definition
CMSIS-SVD	Cortex Microcontroller Software Interface Standard - System View Description; an XML format describing microcontroller hardware peripherals.
DWARF	Standardized debugging data format embedded in ELF binaries that maps machine code to source code lines and symbols.
Hardware Breakpoint (<code>hb</code>)	A breakpoint implemented using dedicated CPU comparator registers, required for debugging code in read-only flash memory.
MMIO	Memory-Mapped Input/Output; a hardware architecture where peripheral registers are mapped into the CPU's regular memory address space.
MSP	Main Stack Pointer; the primary ARM Cortex-M stack pointer register.
OpenOCD	Open On-Chip Debugger; a software bridge that connects GDB to physical hardware via a Debug Probe.
PyCortexMDebug	A GDB Python extension that parses SVD files to provide live peripheral inspection and register manipulation.
Reset Vector	The address stored at offset <code>0x00000004</code> in the vector table that points to the first instruction executed upon CPU reset.
SIO	Single-Cycle I/O; a dedicated RP2350 hardware block providing zero-wait-state GPIO manipulation.
SVD-Loader	A Ghidra script that imports SVD files to create memory blocks, symbol labels, and C structs for decompilation.
SWD	Serial Wire Debug; a two-wire physical debug protocol (SWCLK, SWDIO) used to debug ARM microcontrollers.
XIP	eXecute In Place; running code directly from external flash memory without copying it to RAM first.

Happy Hacking!